Computer Science & Information Systems

Scalable Services – Lab Sheet 1

1. Objective:

We will be developing two services using different technologies. One is using Django and another is using Flask. By the end of this lab sheet we will have basic understanding about creating a small functional service

2. Steps to be performed:

Building a Hello Django application

Initial Set Up:

To begin, navigate to a directory of your choice on your computer. Then, we can create a "hellodjango" folder

```
C:\Users\admin\scalable service>cd hellodjango

C:\Users\admin\scalable service\hellodjango>
```

Create a virtual environment to work on your application, this is done to separate it from other projects on your machine. Then activate the same using the following commands

```
C:\Users\admin\scalable service\hellodjango>python -m venv hello_env

C:\Users\admin\scalable service\hellodjango>hello_env\Scripts\activate.bat

(hello_env) C:\Users\admin\scalable service\hellodjango>
```

Install Django

```
(hello_env) C:\Users\admin\scalable service\hellodjango>pip install django
Collecting django
  Using cached https://files.pythonhosted.org/packages/75/42/f59a8ebf14be6d17438f13042c775f53d3dfa71fff973e4aef64ca89582c/Django-3.1.6-py3-none-any.whl
Collecting asgiref<4,>=3.2.10 (from django)
  Using cached https://files.pythonhosted.org/packages/89/49/5531992efc62f9c6d08a7199dc31176c8c60f7b2548c6ef245f96f29d0d9/asgiref-3.3.1-py3-none-any.whl
Collecting pytz (from django)
  Using cached https://files.pythonhosted.org/packages/70/94/784178ca5dd892a98f113cdd923372024dc04b8d40abe77ca76b5fb90ca6/pytz-2021.1-py2.py3-none-any.whl
Collecting sqlparse>=0.2.2 (from django)
  Using cached https://files.pythonhosted.org/packages/14/05/6e8eb62ca685b10e34051a80d7ea94b7137369d8c0be5c3b9d9b6e3f5dae/sqlparse-0.4.1-py3-none-any.whl
Installing collected packages: asgiref, pytz, sqlparse, django
Successfully installed asgiref-3.3.1 django-3.1.6 pytz-2021.1 sqlparse-0.4.1
WARNING: You are using pip version 19.2.3, however version 21.0.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

(hello_env) C:\Users\admin\scalable service\hellodjango>
```

Create a new Django project called hellodjangoproject making sure to include the period (.) at the end of the command so that it is installed in our current directory.

```
(hello_env) C:\Users\admin\scalable service\hellodjango>django-admin startproject hellodjangoproject .

(hello_env) C:\Users\admin\scalable service\hellodjango>
```

Using dir command we can see the structure of our project

```
(hello_env) C:\Users\admin\scalable service\hellodjango>dir hellodjangoproject
 Volume in drive C is Windows
 Volume Serial Number is 4871-CB51

 Directory of C:\Users\admin\scalable service\hellodjango\hellodjangoproject

18-02-2021  11:13    <DIR>          .
18-02-2021  11:13    <DIR>          ..
18-02-2021  11:13               429 asgi.py
18-02-2021  11:13             3,218 settings.py
18-02-2021  11:13               781 urls.py
18-02-2021  11:13               429 wsgi.py
18-02-2021  11:13                 0 __init__.py
               5 File(s)          4,857 bytes
               2 Dir(s)  293,230,006,272 bytes free

(hello_env) C:\Users\admin\scalable service\hellodjango>
```

The **settings.py** file controls our project's settings, **urls.py** tells Django which pages to build in response to a browser or URL request, and **wsgi.py**, which stands for Web Server Gateway Interface, helps Django serve our eventual web pages. The **manage.py** file is used to execute various Django commands such as running the local web server or creating a new app. Last, but not least, is the **asgi.py** file, new to Django as of version 3.0 which allows for an optional Asynchronous Server Gateway Interface to be run.

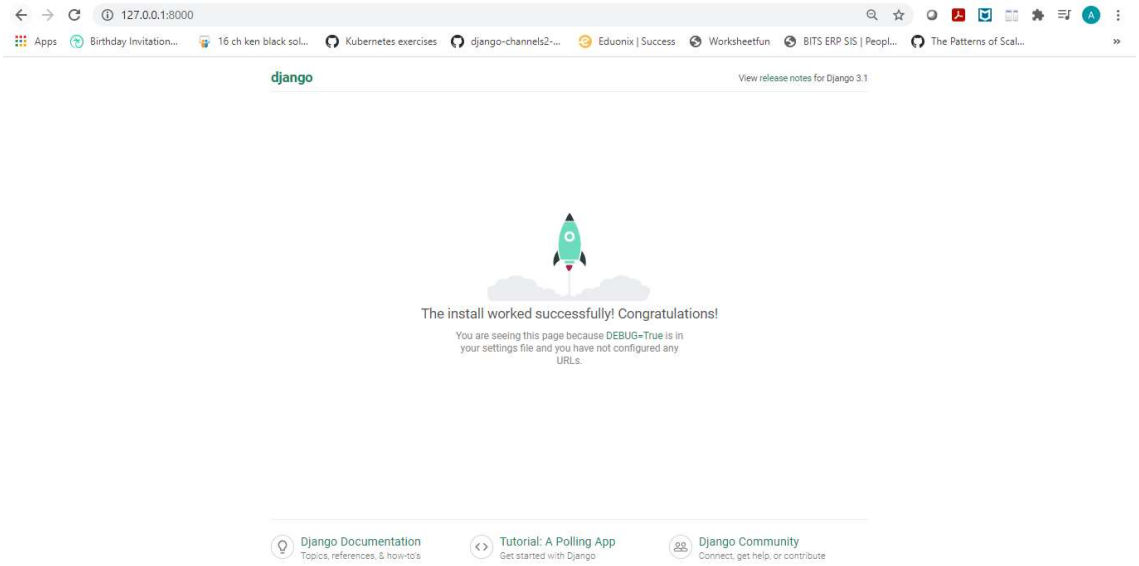
Django comes with a built-in web server for local development purposes which we can now start with the runserver command. But first, use python manage.py migrate to migrate the built-in apps provided for us

```
(hello_env) C:\Users\admin\scalable service\hellodjango>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

```
(hello_env) C:\Users\admin\scalable service\hellodjango>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
February 18, 2021 - 11:26:06
Django version 3.1.6, using settings 'hellodjangoproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

If you visit **http://127.0.0.1:8000/** you should see our familiar Django welcome page.



We can stop the local server with the Control+c command

Create an app:

Django uses the concept of projects and apps to keep code clean and readable. A single Django project contains one or more apps within it that all work together to power a web application.

Now it's time to create our first app. From the command line, quit the server with Control+c. Then use the startapp command followed by the name of our app, which will be hellodjangoapp. We can see the structure of the app using dir command

```
(hello_env) C:\Users\admin\scalable service\hellodjango>python manage.py startapp hellodjangoapp

(hello_env) C:\Users\admin\scalable service\hellodjango>dir hellodjangoapp
 Volume in drive C is Windows
 Volume Serial Number is 4871-CB51

 Directory of C:\Users\admin\scalable service\hellodjango\hellodjangoapp

18-02-2021  11:29    <DIR>          .
18-02-2021  11:29    <DIR>          ..
18-02-2021  11:29                66 admin.py
18-02-2021  11:29               108 apps.py
18-02-2021  11:29    <DIR>          migrations
18-02-2021  11:29                60 models.py
18-02-2021  11:29                63 tests.py
18-02-2021  11:29                66 views.py
18-02-2021  11:29                 0 __init__.py
               6 File(s)            363 bytes
               3 Dir(s)  293,227,458,560 bytes free

(hello_env) C:\Users\admin\scalable service\hellodjango>
```

- admin.py is a configuration file for the built-in Django Admin app
- apps.py is a configuration file for the app itself
- migrations/ keeps track of any changes to our models.py file so our database and models.py stay in sync
- models.py is where we define our database models which Django automatically translates into database tables
- tests.py is for our app-specific tests
- views.py is where we handle the request/response logic for our web app

Even though our new app exists within the Django project, Django doesn't "know" about it until we explicitly add it. In your text editor, open the settings.py file and scroll down to INSTALLED_APPS where you'll see six built-in Django apps already there. Add our new hellodjangoapp app at the bottom:

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'hellodjangoapp',
]
```

URLs, Views, Models, Templates

The first thing that happens within our Django project is a URLpattern is found that matches the homepage. The URLpattern specifies a view which then determines the content for the page (usually from a database model) and then ultimately a template for styling and basic logic. The end result is sent back to the user as an HTTP response.

```
from django.shortcuts import render

# Create your views here.
from django.http import HttpResponse

def homePageView(request):
    return HttpResponse('Hello, Django!')
```

Basically, we're saying whenever the view function homePageView is called, return the text "Hello, Django!"

Now we need to configure our urls. Within the hellodjangoapp, create a new urls.py

```
#hellodjangoapp/urls.py
from django.urls import path
from .views import homePageView

urlpatterns = [
    path('', homePageView, name='home') #new
]
```

Our URLpattern has three parts:

- a Python regular expression for the empty string ''
- a reference to the view called homePageView
- an optional named URL pattern called 'home'

```
# djangohelloproject/urls.py
from django.contrib import admin
from django.urls import path, include #new

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('hellodjangoapp.urls')), # new
]
```

Now whenever a user visits the homepage they will first be routed to the hellodjangoapp and then to the homePageView view.

Hello, Django!

Building a Hello Flask application

We will be using the same environment but new window for command prompt to build our flask application. First create a directory helloflask and make that as the current directory. Make sure for this service as well you activate the same environment variable.

```
(hello_env) C:\Users\admin\scalable service>mkdir helloflask

(hello_env) C:\Users\admin\scalable service>cd helloflask

(hello_env) C:\Users\admin\scalable service\helloflask>
```

Now, install Flask using the pip command

```
(hello_env) C:\Users\admin\scalable service\helloflask>pip install Flask
Collecting Flask
  Using cached https://files.pythonhosted.org/packages/f2/28/2a03252dfb9ebf377f40fba6a7841b47083260bf8bd8e737b0c6952df83f/Flask-1.1.2-py2.py3-none-any.whl
Collecting Jinja2>=2.10.1 (from Flask)
  Using cached https://files.pythonhosted.org/packages/7e/c2/1eece8c95ddbc9b1aeb64f5783a9e07a286de42191b7204d67b7496ddf35/Jinja2-2.11.3-py2.py3-none-any.whl
Collecting itsdangerous>=0.24 (from Flask)
  Using cached https://files.pythonhosted.org/packages/76/ae/44b03b253d6fade317f32c24d100b3b35c2239807046a4c953c7b89fa49e/itsdangerous-1.1.0-py2.py3-none-any.whl
Collecting click>=5.1 (from Flask)
  Using cached https://files.pythonhosted.org/packages/d2/3d/fa76db83bf75c4f8d338c2fd15c8d33fdd7ad23a9b5e57eb6c5de26b430e/click-7.1.2-py2.py3-none-any.whl
Collecting Werkzeug>=0.15 (from Flask)
  Using cached https://files.pythonhosted.org/packages/cc/94/5f7079a0e00bd6863ef8f1da638721e9da21e5bacee597595b318f71d62e/Werkzeug-1.0.1-py2.py3-none-any.whl
Collecting MarkupSafe>=0.23 (from Jinja2>=2.10.1->Flask)
  Using cached https://files.pythonhosted.org/packages/65/c6/2399700d236d1dd681af8aebff1725558cddfd6e43d7a5184a675f4711f5/MarkupSafe-1.1.1-cp37-cp37m-win_amd64.whl
Installing collected packages: MarkupSafe, Jinja2, itsdangerous, click, Werkzeug, Flask
Successfully installed Flask-1.1.2 Jinja2-2.11.3 MarkupSafe-1.1.1 Werkzeug-1.0.1 click-7.1.2 itsdangerous-1.1.0
```

Let's create a package called helloflaskapp, that will host the application. Make sure you are in the helloflask directory and then run the following command:

```
(hello_env) C:\Users\admin\scalable service\helloflask>mkdir helloflaskapp

(hello_env) C:\Users\admin\scalable service\helloflask>
```

The __init__.py for the app package is going to contain the following code:

```
from flask import Flask

app = Flask(__name__)

from helloflaskapp import routes
```

The script above simply creates the application object as an instance of class Flask imported from the flask package. The __name__ variable passed to the Flask class is a Python predefined variable, which is set to the name of the module in which it is used. Flask uses the location of the module passed here as a starting point when it needs to load associated resources such as template files. For all practical purposes, passing __name__ is almost always going to configure Flask in the correct way.
The routes are the different URLs that the application implements. In Flask, handlers for the application routes are written as Python functions, called view functions. View functions are mapped to one or more route URLs so that Flask knows what logic to execute when a client requests a given URL. Here is your first view function, which you need to write in the new module named helloflaskapp/routes.py:

```
from helloflaskapp import app

@app.route('/')
@app.route('/index')
def index():
    return "Hello, Flask!"
```

In this case, the @app.route decorator creates an association between the URL given as an argument and the function. In this example there are two decorators, which associate the URLs / and /index to this function. This means that when a web browser requests either of these two URLs, Flask is going to invoke this function and pass the return value of it back to the browser as a response.

To complete the application, you need to have a Python script at the top-level that defines the Flask application instance. Let's call this script helloflask.py, and define it as a single line that imports the application instance:

```
from helloflaskapp import app
```

The Flask application instance is called app and is a member of the helloflaskapp package. The from helloflaskapp import app statement imports the app variable that is a member of the helloflaskapp package.

3. Outputs/Results:

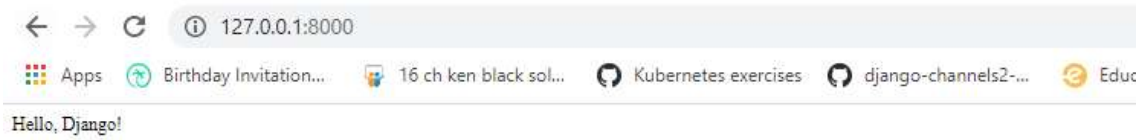
Django App

We have all the code we need now. To confirm everything works as expected, restart our Django server:

```
(hello_env) C:\Users\admin\scalable service\hellodjango>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
February 18, 2021 - 12:20:12
Django version 3.1.6, using settings 'hellodjangoproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

If you visit **http://127.0.0.1:8000/** you should see our page.



Flask App

Before running it, though, Flask needs to be told how to import it, by setting the FLASK_APP environment variable:

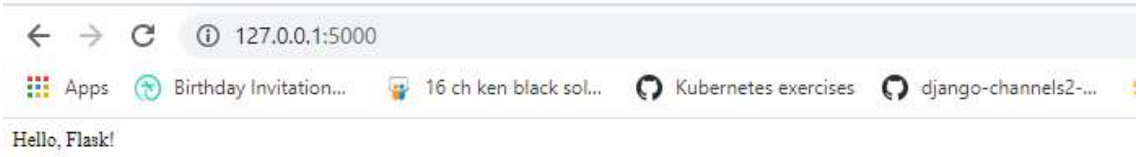
```
(hello_env) C:\Users\admin\scalable service\helloflask>set FLASK_APP=helloflask.py

(hello_env) C:\Users\admin\scalable service\helloflask>
```

You can run your first web application, with the following command:

```
(hello_env) C:\Users\admin\scalable service\helloflask\helloflaskapp>flask run
 * Serving Flask app "helloflask.py"
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

If you visit **http://127.0.0.1:5000/** you should see our page.



When you are done playing with the server you can just press Ctrl-C to stop it.

4. Observations:

You can notice that the services are up and running on their respective ports. Django service is running at 8000 port and Flask is running at 5000 port

5. References:

https://docs.djangoproject.com/en/3.2/
https://flask.palletsprojects.com/en/2.0.x/quickstart/